

Programming Assignment 1

CSCI 4061 Summer 2026

Important Dates

- **Release Date:** June 17, 2026 11:59 PM
- **Phase 1 Due Date:** June 28, 2026 11:59 PM
- **Phase 2 Due Date:** July 9, 2026 11:59 PM

Objective

Develop a program that summarizes and logs the extent of file corruption in a collection of files within a given directory. File corruption is defined as the presence of non-ASCII characters.

The analysis should be done using multiple processes. Each process analyzes a single file to count the number of corrupted characters and the number of lines where corruption has occurred. It then reports the results back to the main process. The main process summarizes the aggregate data and prints the total corrupted character and line counts across the directory to the terminal. This kind of task is common in real-world applications such as data sanitization, security, and log auditing.

Advisory

Make sure each phase of the project is tested on CSE lab machines running Linux before submission since some system calls may not be supported on other environments.

Tasks

The assignment is split into two phases with different due dates listed above. Each phase is further broken down into tasks. There are five tasks with detailed descriptions below. Task 1, Task 2, and Task 3 are part of Phase 1 and should be completed before the due date listed above for Phase 1. There is no interprocess communication in Phase 1, and the directory traversal is strictly shallow. Task 4 and Task 5 are part of Phase 2 and should be completed by the due date listed for Phase 2. In Phase 2, you will upgrade your directory parsing to handle recursive traversal and implement interprocess communication.

Task 1: Argument Parsing & Directory Extraction

The main process should be able to receive the path of the directory from the command line in the following format:

```
./logger PATH_TO_DIR
```

The main process should then extract a list of all files in the given directory.

Task 2: Child Process Creation/Termination

The main process should create child processes to handle files. The number of child processes created by the main process should be equal to the total number of files discovered in the directory. The implementation of the child process should be in a separate C source code file called `counter.c` which is different from the source file of the main process.

The child process should be able to receive the path to the file it is supposed to validate via command line arguments. After forking each child process, it should call the `exec1(...)` function to execute the code in the child process. The main process is also responsible for handling the termination of child processes, so it can only exit when all the child processes have terminated.

Task 3: Extract Data from Files

The child process should read its assigned file to identify and count instances of file corruption. It must track two specific metrics: the total number of corrupted characters, and the total number of lines that contain at least one corrupted character.

Each child process needs to write out its own results into a file using the same name as the original file it reads from, but with a `.log` extension. For example, if the original file is named `file1.txt`, the child process should create a file called `file1.log` and write exactly two lines in the file: the first line should be the total number of corrupted characters, and the second line should be the total number of lines where corruption occurred.

Task 4: Recursive Directory Traversal (Phase 2 Only)

Make a copy of the code you completed in tasks 1, 2 & 3 and use it as a starting base for Phase 2.

Upgrade your directory parsing logic from Task 1. When reading a directory, your **main process** must distinguish between a regular file and a subdirectory. If it encounters a subdirectory, the main process must recursively traverse into it to evaluate any files inside at any depth.

Note: You should NOT create a deep tree of child processes. The single main process should recursively traverse the directories to find all files, and then spawn a flat list of child processes to handle each discovered file.

Task 5: Inter-Process Communication with Pipes (Phase 2 Only)

Child processes DO NOT need to write results to a file. Remove the parts of the code where you write the results to `.log` files. Instead, each child process sends its results directly back to the single main process through a pipe and terminates. The message sent back to the main process must include two data points: the corrupted character count and the corrupted line count for the file. The main process handles outputting the results, and the format is specified in the Expected Output section.

Useful APIs

You may use but are not limited to the APIs mentioned below for the project:

- **Processes:** `fork()`, `exec()`, `wait()`/`waitpid()`.

- **System File I/O:** `open()`, `close()`, `read()`, `write()`.
- **Standard File I/O:** `fopen()`, `fclose()`, `fread()`, `fwrite()`, `fscanf()`, `fprintf()`, `fgets()`, `fputs()`, `getline()`.
- **Directory:** `mkdir()`, `opendir()`, `readdir()`, `closedir()`.
- **File Stats (Phase 2):** `stat()`, `lstat()`, `S_ISDIR()`, `S_ISREG()` (requires `<sys/stat.h>`).
- **Pipes:** `pipe()`.
- **Character Types:** `isascii()` (requires `<ctype.h>`).

Assumptions/Notes

Please adhere to the following assumptions:

- The code execution commands will be run at the same level as the given **Makefile**.
- The maximum path length will be 1024.

Expected Output

The program should only output the following lines as the result. *Note: Because child processes run concurrently, the order in which the filenames are printed does not matter. For Phase 2, the filename printed should include its relative path if it is inside a subdirectory.*

Example Input Directory Structure:

```
testcase_folder/
|- file1.txt
|- subfolderA/
|   |- file2.txt
|- subfolderB/
|- file3.txt
```

Example Expected Output:

```
testcase_folder/file1.txt: 0, 0
testcase_folder/subfolderA/file2.txt: 5, 2
testcase_folder/subfolderB/file3.txt: 1, 1
Total Corrupted Characters: 6
Total Corrupted Lines: 3
```

Please do not include any other output in your final submission.

Compilation and Execution

We have provided you with a **Makefile** that contains the necessary rules to build the target program and also clean them. To compile the entire code, use the following command:

```
make all
```

After compilation, run the program with the target directory as an argument:

```
./logger PATH_TO_DIR
```

To clean the compiled executables and object files, use:

```
make clean
```

Please **DO NOT** add or delete source code files in the skeleton code we provided, or make changes to the Makefile.

Testing

There will be one set of testcases for this project. Testcases 1-5 will be used in both phases. Testcase 6 will only be used to evaluate Phase 2.

- **Testcase 1:** An empty directory.
- **Testcase 2:** A directory with a single nonempty file containing no corruption.
- **Testcase 3:** A directory with several files, including an empty file and a heavily corrupted file.
- **Testcase 4:** A directory with 5 files, containing a mix of pure ASCII and corrupted characters.
- **Testcase 5:** A directory with 15 files, none of which are empty, representing various states of file corruption.
- **Testcase 6 (Phase 2 Only):** A directory containing multiple levels of nested subdirectories, with files distributed at various depths.

Submission

For each phase, create a README file that answers mainly the following:

1. Team members.
2. x500.
3. Division of task across members.
4. Lab machine used for testing purposes.
5. Design specifications: Mention your project design in detail and mention any challenges faced.
6. Document and explain any usage of AI tools.
7. Anything the TA should know about.

Submit a single `.tar.gz` file containing your README and source code to Gradescope (`pa1ph1.tar.gz` for Phase 1, and `pa1ph2.tar.gz` for Phase 2). Only one member of a group should submit to Gradescope. Make sure to add your group members' names in Gradescope at the time of submission.

AI Usage

Large Language Models, such as ChatGPT or CoPilot, may be used in a limited manner for programming assignments with appropriate citation. For programming assignments, you may use AI tools to help with some basic helper function code (similar to library functions). You must NOT use these tools to design your solution, or to generate a significant part of your assignment code. You must design and implement the code yourself. You must clearly identify parts of the code that you used AI tools for, providing a justification for doing so. You must understand such code and be prepared to explain its functionality. Note that the goal of this assignment is to provide you with a deeper and hands-on understanding of the concepts you learn in the class, and not merely to produce “code that works”. If you are in doubt as to whether you are using LLMs appropriately in this course, please discuss your situation with the instructor or TA. Examples of citing AI tools are available at: libguides.umn.edu/chatgpt. You are responsible for fact-checking statements and correctness of code generated by LLMs.

Rubrics

Phase 1: 50 points

- `exec*` usage: 5 points.
- Process spawning: 5 points.
- File handling: 5 points.
- README: 5 points.
- Provided test cases (1-5): 25 points.
- Hidden test cases: 5 points

Phase 2: 50 points

- Pipes: 10 points.
- Directory Traversal (recursive): 10 points.
- Provided Testcases (1-6): 25 points.
- Hidden test cases: 5 points